BLOQUE 1 - SEGUNDA PARTE: SISTEMAS DE BASES DE DATOS E INTRODUCCIÓN AL MODELO RELACIONAL

1. Sistemas de bases de datos

2. Ventajas de los sistemas de bases de datos respecto de otros elementos tradicionales de almacenamiento

3. Elementos de un sistema de base de datos

4. Arquitectura en tres niveles o referencia ANSI-SPARC.

5. El diccionario de datos

6. Funcionamiento de un SGBD

7. Lenguaje SQL como medio de acceso a los sistemas gestores de bases de datos: LDD y LMD

8. Modelo relacional: Estructura e integridad

9. Estructura de datos relacional

10. Integridad de los datos

11. Arquitectura de una base de datos relacional

RAs y CE trabajados y evaluados en el bloque:

RA	CE
1	a, b, c, d, e, f
2	a, b, c, d

1. Sistemas de bases de datos

Como hemos visto en la introducción al tema, podemos definir una base de datos como **"Un conjunto de datos estructurados y relacionados entre sí, los cuales pueden ser manipulados y consultados por un sistema de información normalmente vinculado a una empresa o negocio".**

Una base de datos también se puede considerar como un modelo de una actividad o problema del mundo real, ya que permite describir los componentes y sus características que intervienen en un problema o situación real y la forma en que se

relacionan estos elementos entre sí.

2. Ventajas de los sistemas de bases de datos respecto de otros elementos tradicionales de almacenamiento

Como también hemos visto en la primera parte del tema, los sistemas de bases de datos aportan muchas ventajas respecto a los sistemas de ficheros clásicos. Estas son las principales:

Independencia entre datos y aplicaciones.
Evitar redundancia de datos
Evitar inconsistencia o incoherencia de datos
Centralizar el control de la BBDD
Mejorar la accesibilidad de los datos
Preservar la integridad de los datos
Garantizar la confidencialidad y la seguridad de los datos
Acceso concurrente a los datos

Actividad 1: Rellena la siguiente tabla, justificando las ventajas expuestas.

Independencia entre datos y aplicaciones	
Evitar redundancia de datos	
Evitar inconsistencia o incoherencia de datos	
Centralizar el control de la BBDD	
Mejorar la accesibilidad de los datos	

Preservar la integridad de los datos	
Garantizar la confidencialidad y seguridad de los datos	
Accesos concurrentes a los datos	

3. Elementos de un sistema de base de datos

Los componentes principales de un SGBD son:

Los **datos** que son almacenados en la base de datos.

El **diccionario de datos o catálogo del sistema:** Es una base de datos de las bases de datos almacenadas en él. Es donde el Sistema Gestor de Bases de Datos guarda los **metadatos** (es decir, la información que describe a los propios datos). En lugar de guardar la información de una empresa, por ejemplo (clientes, productos, proveedores, …) el diccionario de datos guarda la **estructura de la base de datos.**

Dentro de esta información podemos encontrar, por ejemplo:

¿Qué **tablas** existen? (ej. CLIENTES, FACTURAS).

¿Qué **columnas** tiene cada tabla y de qué **tipo** son? (ej. el campo edad es un número entero, email es texto).

¿Cuáles son las **claves primarias y foráneas** que unen las tablas?

¿Quién tiene **permiso** para ver o cambiar la información?

Sistema de gestión de la BD (SGBD): Conjunto de programas que gestionan la BD. Las funciones principales son crear, actualizar, consultar, administrar y proteger la BD.

Administrador de la base de datos (ABD): Función que pueden llevar a cabo una o varias personas. Entre sus responsabilidades están la de creación, gestión y mantenimiento de la BD.

Usuarios: Todos aquellos que utilizan la BD. Distinguiendo entre usuarios

informáticos o técnicos que pueden acceder directamente a los datos almacenados e incluso manipularlos y los no informáticos (usuarios finales) a los que va dirigida la BBDD.

Actividad 2: Crea una infografía o búscala por internet que muestre la interacción de los elementos de la base de datos mostrados.

Actividad 3: "La Biblioteca Escolar y el Sistema de Información"
En la biblioteca de un instituto, se gestionan los préstamos, los libros y las sanciones, cuenta con un sistema de información. A continuación, se describen diferentes elementos y roles que intervienen en su funcionamiento diario:
1. El **catálogo general (fichero)** donde se indica cuántas estanterías existen, la estructura del código de barras de cada libro, qué campos forman la ficha de un socio (nombre, DNI, teléfono) y qué reglas se aplican (por ejemplo, "un alumno no puede llevarse más de 3 libros a la vez").
2. El **personal técnico** del departamento de informática que se encarga de instalar el software, crear las cuentas, dar permisos, definir las normas de acceso y hacer copias de seguridad de las fichas del sistema.
3. Las **fichas** concretas guardadas en el sistema, como por ejemplo: *"Cien años de soledad"*, *"María López García"*, el valor *"978-84-376-0494-7"* o la fecha de devolución *"2026-10-15"*.
4. El **alumnado** que consulta la disponibilidad de un libro desde la web, el **profesorado** que solicita lecturas recomendadas y la **administrativa** de la biblioteca que registra la entrega de un ejemplar en la aplicación.
Relaciona cada uno de los 4 números del enunciado anterior con su concepto correspondiente dentro del mundo de los **Sistemas de Bases de Datos (SGBD)**:

- **[A]** Datos
- **[B]** Diccionario de Datos (o Catálogo del Sistema)
- **[C]** Administrador de la Base de Datos (DBA)
- **[D]** Usuarios (Usuarios finales / Aplicaciones)

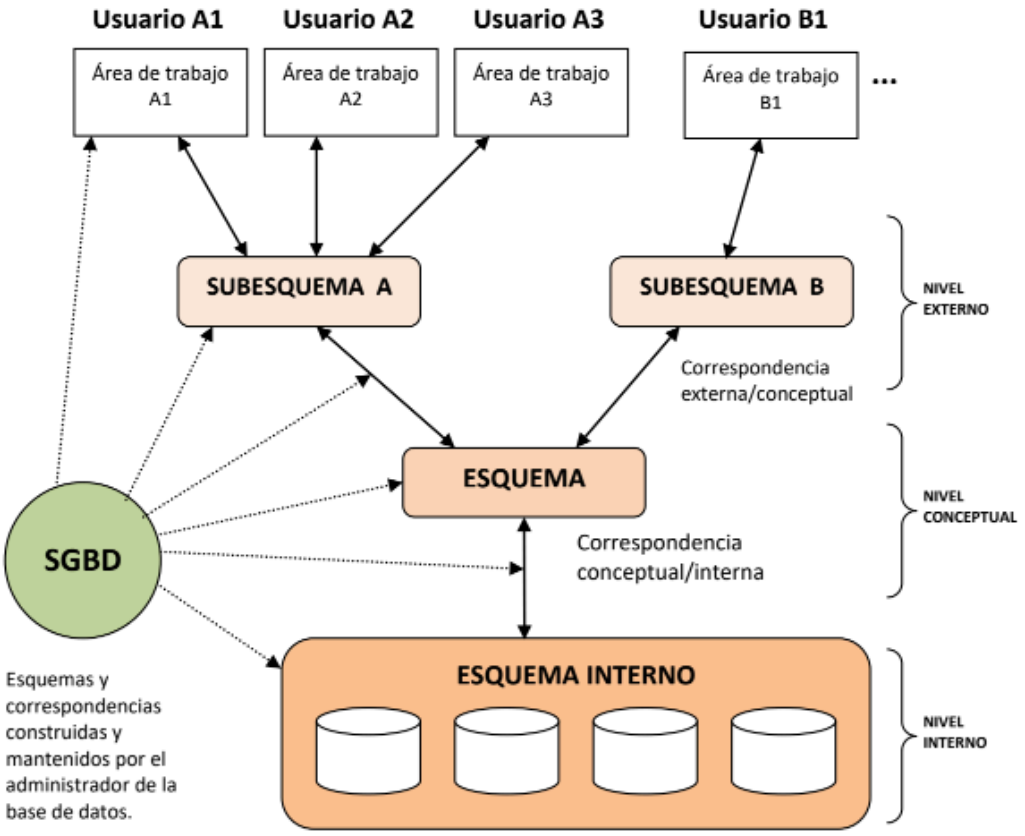
Justifica brevemente tu respuesta explicando el porqué de cada relación.

Rol	Concepto	Justificación técnica

1		
2		
3		
4		

3. Arquitectura en tres niveles o referencia ANSI-SPARC.

Uno de los objetivos de un sistema de bases de datos es proporcionar a los usuarios una visión abstracta de la información, ocultando ciertos detalles complejos acerca de cómo se almacenan los datos, pero permitiendo una recuperación eficaz de la información. Este proceso se realiza a través de varios niveles de abstracción.
Las arquitecturas de bases de datos han evolucionado mucho desde sus comienzos, pero todas siguen, en mayor o menor medida, las recomendaciones del estándar descrito por el comité **ANSI/X3/SPARC** (Standard Planning and Requirements Committee of the American National Standards Institute – Comité de Planificación y Requerimientos de estándares del Instituto Nacional de Estándares de Estados Unidos, división X3), que data de finales de los años setenta. Este comité propuso una **arquitectura general para los SGBD basada en tres niveles de abstracción o esquemas: el nivel físico, o de máquina, el nivel externo, o de usuario, y el nivel conceptual**. Asimismo describió las interacciones entre estos tres niveles y todos los elementos que conforman cada uno de ellos.
A continuación se muestra la organización de la arquitectura ANSI/X3/SPARC:



Nivel externo

Es el más cercano a los usuarios. Representa la visión o acceso de cada tipo de usuario de la base de datos.

En este nivel, el administrador de la base de datos se encarga de definir los subconjuntos de datos a los que cada perfil de usuario debe tener acceso. Por cada tipo de usuario se especifica un **esquema externo, subesquema o vista externa**. Por lo tanto, existirán tantos subesquemas como tipos de usuarios tenga la BBDD. Habrá usuarios que podrán acceder a más de un esquema externo, y un esquema externo podrá ser compartido por varios usuarios.

Nivel conceptual

Es una representación de toda la información contenida en la BBDD en una forma más abstracta que la contenida en la estructura interna. Se puede considerar como un modelo teórico de la base de datos sobre el cual estarán asentados los submodelos externos.

El administrador es el único que trabaja a nivel conceptual, ya que los usuarios sólo trabajan a nivel externo utilizando subconjuntos de la estructura conceptual.

El objetivo de este nivel es proporcionar una descripción real de la empresa independiente de la estructura física de los datos. El nivel conceptual se describe por medio de un **esquema o vista conceptual de la BBDD**. El esquema conceptual contendrá:

- Las **entidades** u objetos del mundo real a almacenar (empleados, departamentos, ventas, …)
- Los **atributos de las distintas entidades** (por ejemplo, para la entidad empleado: Identificador de empleado, nombre, dirección, DNI, nº hijos, sueldo etc.)
- **Relaciones entre las distintas entidades** (Por ejemplo: Un empleado pertenece a un departamento, un departamento contiene varios empleados, etc.)
- **Verificaciones de integridad** (Por ejemplo: un empleado no podría pertenecer a un departamento que no existe).

Nivel interno

Es el nivel más bajo de la arquitectura, el que está **pegado al almacenamiento** en el servidor. Aquí ya no pensamos en la información por ejemplo de los clientes sino en cómo esta se guarda físicamente.

En este nivel, el SGBD decide y organiza:

- **El formato de almacenamiento:** Cuántos bytes exactos ocupa cada dato en el disco duro o en la memoria del servidor.
- **La organización del archivo:** Cómo se ordenan los archivos de la base de datos dentro del sistema operativo.
- **Las rutas de acceso rápido (Índices):** Creación de "índices" (como el índice de un libro) o listas de punteros para encontrar un dato en milisegundos sin tener que leer todo el disco.
- **El empaquetado de datos:** Cómo agrupar varios datos juntos para leerlos de golpe y hacer que el acceso a la información sea más rápido.

El SGBD debe poder garantizar la **transferencia de los datos desde el nivel interno al nivel externo**, a este proceso se llama **transformación de datos o mapeo (data mapping)**. Para ello existen dos niveles de correspondencia:

Correspondencia conceptual/interna: Permite el paso de la vista conceptual a la

vista interna, y viceversa. Especifica cómo se representan los registros y campos conceptuales en el nivel interno. Si se modifica la estructura interna de la base de datos, la correspondencia conceptual/interna deberá modificarse para que no varíe el esquema conceptual. De este modo se conserva la independencia de los datos.
Correspondencia externa/conceptual: Permite el paso de una vista externa específica a la vista conceptual, y viceversa.
Los subesquemas, el esquema conceptual, el esquema interno y las correspondencias conceptual/interna y externa/conceptual, las describe el administrador de la base de datos y quedan almacenados dentro del diccionario de la BD para futuras consultas del SGBD.
Cuando un usuario desea acceder a la base de datos, el SGBD examina el diccionario de datos para comprobar si la solicitud puede ser realizada y el modo de realizar las transformaciones pertinentes de los datos.
El **objetivo de la arquitectura de tres niveles es separar los programas de**

aplicación de la base de datos física.
La mayoría de los SGBD no distinguen del todo los tres niveles, de hecho, algunos incluyen detalles del nivel físico en el esquema conceptual.
Hay que destacar que los tres esquemas no son más que descripciones de los mismos datos pero con distintos niveles de abstracción. Los únicos datos que existen realmente están a nivel físico, almacenados en un dispositivo de almacenamiento.

Actividad 4: Identificar los niveles de la arquitectura ANSI/SPARC en una app.
En esta actividad vamos a comprender la separación de niveles en la arquitectura ANSI/SPARC de una Base de Datos (Nivel Externo, Nivel Conceptual y Nivel Interno) a través de un ejemplo real y cotidiano.
Imagina la **base de datos de una plataforma de películas y series en streaming.** Dentro de esta plataforma existen diferentes tipos de acceso a la misma información:
1. El Usuario final (Cliente): Abre la app en su móvil. Solo ve portadas de películas recomendadas para él, su lista de favoritos y el botón de "Reproducir". No ve ni le importa el precio que pagó la plataforma por los derechos de la película ni los datos

bancarios de otros usuarios.

2. El Departamento de Contabilidad: Accede a una pantalla donde solo ve el catálogo de películas con sus costes de licencia, caducidad de contratos y las métricas de reproducciones totales para calcular los pagos.

3. El Diseñador de la Base de Datos: Ve la estructura completa de la información: la tabla de USUARIOS, la tabla de PELÍCULAS, la tabla de SUSCRIPCIONES y cómo se relacionan entre sí mediante sus claves primarias y foráneas, con sus nombres de campo y restricciones.

4. El Servidor y el Disco SSD: En los servidores en la nube, el motor de la base de datos guarda estas tablas en archivos de disco en formato binario. Algunas de las normas que se siguen para almacenar esta información, son las siguientes: se reservan 8 bytes para las fechas, 4 bytes para los IDs numéricos, utiliza índices especiales (Árboles B+) para que la búsqueda de una película tarde menos de 0,01 segundos y aplica compresión de datos para ahorrar espacio.

Dada esta información, responde a las siguientes preguntas:

Parte 1: Identificación de Niveles

Relaciona cada una de las siguientes descripciones con el nivel de la arquitectura ANSI/SPARC al que pertenece (**Externo**, **Conceptual** o **Interno**):

- **a)** La consulta SQL que crea la tabla `PELICULA` definiendo los campos `id_pelicula INT, titulo VARCHAR(100)` y la restricción de que la duración debe ser mayor a cero.
- **b)** La pantalla de la app del móvil donde un usuario ve su lista personalizada de "Seguir viendo".
- **c)** La estructura en un disco SSD que guarda las páginas de datos en bloques de 8 KB y utiliza un índice B-Tree para encontrar el título de una película al instante.
- **d)** El panel de administración que utiliza el equipo de soporte al cliente para ver los datos de facturación de un abonado.

Parte 2: Análisis y Reflexión

1. ¿Por qué es útil la **separación entre el Nivel Externo y el Nivel Conceptual**? ¿Qué pasaría si el usuario final pudiera ver toda la base de datos directamente tal y como está guardada?

2. Si el Administrador del Sistema decide cambiar el disco duro por uno más rápido o cambiar el formato de compresión del archivo (Nivel Interno), ¿debería esto cambiar la forma en la que el usuario ve sus películas en la app (Nivel Externo)? Justifica tu respuesta.

Actividad 5: ANSI/SPARC en PostgreSQL

En esta actividad vamos a ver cómo PostgreSQL gestiona los tres niveles y cómo esta arquitectura protege a los usuarios ante cambios en la estructura física.

Imagina que gestionamos un **taller mecánico**. Vamos a crear la tabla base donde se almacena la información de los vehículos y sus costes de reparación.

```
-- Creación de la estructura global (Nivel Conceptual)
CREATE TABLE vehiculos (
    id_vehiculo SERIAL PRIMARY KEY,
    matricula VARCHAR(10) UNIQUE NOT NULL,
    modelo VARCHAR(50) NOT NULL,
    coste_reparacion DECIMAL(10,2) NOT NULL,
    dni_propietario VARCHAR(9) NOT NULL
);
-- Inserción datos de prueba
INSERT INTO vehiculos (matricula, modelo, coste_reparacion,
dni_propietario)
VALUES
('1234ABC', 'Seat Ibiza', 150.50, '12345678A'),
('5678DEF', 'Ford Focus', 300.00, '87654321B');
```

Pregunta 1: Explica qué hace cada línea de código identificando los tipos de datos y las restricciones. ¿Por qué si la tabla tiene 5 campos solo inserto 4?

Pregunta 2: Abre la herramienta de consultas (**Query Tool**) en pgAdmin 4 y escribe y ejecuta el script anterior para definir la estructura conceptual. Haz captura de pantalla del resultado.

Pregunta 3: ¿Qué elementos del **Nivel Conceptual** identificas en las instrucciones SQL anteriores? *(Estructuras, tipos de datos, restricciones)*.

Pregunta 4: Nivel Externo (Vistas Adaptadas)

El recepcionista del taller necesita ver los coches para informar a los clientes, pero

por privacidad no debe ver el coste interno de la reparación ni el DNI de otros clientes. Ejecuta el siguiente código para crear una **vista (Vista Externa)** para el recepcionista:

```
-- Creación del Nivel Externo
CREATE VIEW vista_recepcion AS
SELECT matricula, modelo
FROM vehiculos;

-- Consultamos desde la perspectiva del recepcionista
SELECT * FROM vista_recepcion;
```

¿Qué hace cada línea de código?Haz captura de pantalla del resultado

Pregunta 5: Al ejecutar **`SELECT * FROM vista_recepcion;`**. ¿Aparecen los campos `coste_reparacion` o `dni_propietario`? ¿Por qué esta separación demuestra la utilidad del Nivel Externo?

Pregunta 6: El Nivel Interno (Catálogo del Sistema y Almacenamiento). Ahora vamos a "abrir el capó" de PostgreSQL y ver cómo el motor guarda esto físicamente en el servidor.

```
-- Consultar el nombre del archivo físico en el disco duro del servidor
SELECT relname, pg_relation_filepath(oid) AS
ruta_archivo_fisico
FROM pg_class
WHERE relname = 'vehiculos';
```

Ejecuta las instrucciones, haz captura de pantalla del resultado, explica qué hace cada una de las líneas del código y anota la ruta del archivo físico que te ha devuelto PostgreSQL (por ejemplo: `base/16384/24581`).

Pregunta 7: Ejecuta la siguiente instrucción:

```
-- Crear un índice (Estructura interna de aceleración B-Tree)
CREATE INDEX idx_matricula ON vehiculos(matricula);
```

¿Qué hace esta instrucción? Haz captura de pantalla del resultado.

Al crear el índice `idx_matricula`, ¿has tenido que modificar la tabla original o la vista del recepcionista? ¿Qué propiedad de la arquitectura ANSI/SPARC se está

cumpliendo aquí?

Pregunta 8: Comprobación de la Independencia de Datos. Añadiremos una nueva columna confidencial a la tabla base (cambio en el Nivel Conceptual):

-- Modificamos el Nivel Conceptual

```
ALTER TABLE vehiculos ADD COLUMN observaciones_mecanico TEXT;
```

¿Qué hace la orden?

Haz captura de pantalla del resultado y captura de pantalla del Object Explorer (izquierda de la pantalla) donde se vea la tabla con el nuevo campo.

Pregunta 8: Vuelve a ejecutar la consulta del recepcionista: `SELECT * FROM vista_recepcion;`

¿Ha fallado la vista del recepcionista o se ha visto afectada por añadir la nueva columna a la base de datos? Explica la relación con la **Independencia Lógica de Datos**.

Actividad 6: Elección de tipos de datos en SQL (PostgreSQL)

Busca información sobre los tipos de datos más utilizados en PostgreSQL (puedes consultar la documentación oficial):

https://www.postgresql.org/docs/current/datatype.html

a) Crea una tabla como la siguiente agrupándolos por categorías.

Categoría	**Tipo de Dato (SQL)**	**Características / Rango / Tamaño**	**Ejemplo de uso**
Numéricos			
Entero	INT / INTEGER	Entero estándar de 4 bytes (de $-2.147.483.648$ a $+2.147.483.647$).	Stock de un producto (45).
Entero Pequeño			
Numérico Autoincremental			
Numérico Decimal Exacto			

Numérico Decimal Flotante			
Texto			
Texto de longitud fija			
Texto de longitud variable limitada			
Texto de longitud ilimitada			
Fecha y hora			
Fecha			
Hora			
Fecha y Hora con zona horaria			
Valores lógicos			
Lógico / Booleano			
Otros tipos de datos			
Binario / Datos complejos			
JSON / Estructurado			

b) Busca en la documentación https://www.postgresql.org/docs/current/datatype.html otros tipos de datos que nos pueden ser útiles para almacenar ciertos tipos de información que no se encuadra en los tipos vistos en la tabla. Elige **al menos 10** y para cada uno de ellos especifica el nombre, qué información almacena, sus características y un ejemplo de uso.

c) Determina para los siguientes atributos que nos podemos encontrar en cualquier base de datos real, cuál sería el tipo de datos más adecuado. Justifica tu elección teniendo en cuenta el espacio en disco y las operaciones que se realizarán con él (por ejemplo: *"¿se van a hacer cálculos matemáticos o solo es una cadena de texto?"*).
MUY IMPORTANTE: "Si un campo solo contiene números pero NO se van a hacer operaciones matemáticas con él (sumar, promedio), se debe guardar como TEXTO (VARCHAR / CHAR)." (Ejemplo: DNI, Teléfono, Código Postal, IBAN).

1. dni_nie: Documento de identidad de un usuario (ej. "12345678A").
2. telefono: Número de teléfono móvil de contacto (ej. "+34612345678").
3. nombre_persona: Nombre de un cliente o empleado.
4. edad: Edad de un usuario en años.
5. salario_mensual: Sueldo de un empleado con dos decimales.
6. sexo_biologico: Sexo indicado en el registro (ej. 'M', 'F').
7. es_socio: Estado de la suscripción de un cliente (si es socio activo o no).
8. fecha_nacimiento: Día, mes y año de nacimiento.
9. hora_fichaje: Momento exacto de entrada al trabajo de un empleado (sin fecha).
10. fecha_hora_pedido: Registro temporal del momento exacto en que un usuario hace un pedido online.
11. observaciones: Comentarios abiertos sobre una incidencia (de longitud variable e impredecible).
12. codigo_postal: Código postal de una localidad en España (ej. "28001").
13. iban_cuenta: Código de cuenta bancaria internacional (ej. "ES9121000418450200051332").
14. precio_producto: Precio de venta de un artículo en un catálogo e-commerce.
15. descuento_porcentaje: Porcentaje de descuento aplicado (ej. 15.5 para un $15,5\%$).
16. temperatura_sensor: Lectura continua de temperatura enviada por un sensor IoT (ej. 23.4567).
17. id_autoincremental: Clave primaria secuencial generada automáticamente para una tabla de facturas.

18. num_matricula_coche: Matrícula de un vehículo (ej. "1234BBB").
19. foto_perfil: Archivo de imagen o datos binarios subidos directamente a la BD.
20. matriz_preferencias_json: Configuración de interfaz del usuario guardada en formato JSON.
21. ip_usuario: Dirección IP de conexión (ej. "192.168.1.100" o versión IPv6).
22. peso_kg: Peso corporal de un paciente registrado en una clínica.
23. stock_almacen: Unidades disponibles de un producto en stock (no admite negativos ni decimales).
24. codigo_hash_password: Contraseña cifrada guardada mediante algoritmo SHA-256 (longitud fija de 64 caracteres hex).
25. color_hex: Código de color Web en formato hexadecimal (ej. "#FF5733").
26. anno_fabricacion: Año de construcción de un vehículo o vivienda (ej. 2024).
27. distancia_km: Distancia GPS en kilómetros entre dos ciudades.
28. numero_seguridad_social: Identificador de la Seguridad Social de un trabajador.
29. talla_ropa: Talla de una prenda de vestir (ej. 'XS', 'S', 'M', 'L', 'XL').
30. duracion_segundos: Tiempo total de duración de una canción o vídeo expresado en segundos.

Muestra el resultado en una tabla como la siguiente:

#	**Atributo**	**Tipo Sugerido (PostgreSQL)**	**Justificación técnica**
1	**dni_nie**	VARCHAR(9) o CHAR(9)	Cadena alfanumérica fija de 9 caracteres. No es un número matemático.
2	**telefono**		
3	**nombre_persona**		
4	**edad**		
5	**salario_mensual**		
6	**sexo_biologico**		

7	**es_socio**		
8	**fecha_nacimiento**		
9	**hora_fichaje**		
10	**fecha_hora_pedido**		
11	**observaciones**		
12	**codigo_postal**		
13	**iban_cuenta**		
14	**precio_producto**		
15	**descuento_porcentaje**		
16	**temperatura_sensor**		
17	**id_autoincremental**		
18	**num_matricula_coche**		
19	**foto_perfil**		
20	**matriz_preferencias**		
21	**ip_usuario**		
22	**peso_kg**		
23	**stock_almacen**		

24	**codigo_hash_password**		
25	**color_hex**		
26	**anno_fabricacion**		
27	**distancia_km**		
28	**numero_seguridad_social**		
29	**talla_ropa**		
30	**duracion_segundos**		

Actividad 7: Crea una base de datos PRUEBA con los campos que se indican.

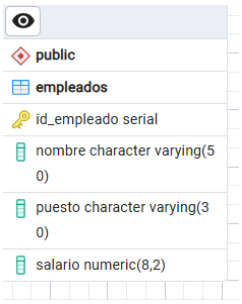
Tabla socios	
CodSocio	Texto(10)
Nombre	Texto(20)
Apellidos	Texto(30)
Domicilio	Texto(30)
Población	Texto(30)
CodPostal	Texto(5)
Provincia	Texto(15)
Teléfono	Texto(9)
Observaciones	Texto Largo
Sexo	Texto(1)

Tabla películas	
IdPelicula	Autonumérico
Título	Texto(40)
Protagonista	Texto(30)
Secundario	Texto(30)
Director	Texto(30)
Nacionalidad	Texto(30)
IdTema	Entero largo
Año	Entero
Duración	Entero
Precio	Moneda

Tabla temas	
IdTema	Autonumérico.
Tema	Texto(15)

Tabla alquiler	
IdAlquiler	Autonumérico
CodSocio	Texto(10)
IdPelicula	Entero largo
FechaAlquiler	Fecha
FechaDevolucion	Fecha
Devuelto	Sí/no

Con la herramienta **ERD for database**, visualiza el modelo de tablas y haz captura de pantalla. Las tablas tendrán una estructura similar a la siguiente:



La herramienta ERD Tool solo nos sirve para hacer o mostrar un diseño gráfico, pero los cambios visuales que se realizan en el diagrama no tienen efecto en la base de datos real. Para aplicar los cambios en la base de datos debemos:

Generar/Ejecutar el script SQL :

- En la barra de herramientas del editor ERD, debes pulsar el botón "Generate SQL" (o el icono de la consola de comandos).
- Esto abrirá la herramienta de consultas (Query Tool) con el código SQL (CREATE TABLE, ALTER TABLE, etc.) generado a partir de tu diagrama.
- Debes revisar el código SQL y hacer clic en Execute (F5) para que las instrucciones se ejecuten en la base de datos real.

Guardar el archivo de proyecto (.pgerd):

- El botón de guardar dentro de la herramienta ERD (Ctrl + S o el icono del disquete) **solo guarda el archivo de diseño del esquema**, que es un archivo propio con la maquetación y estructura del diagrama para volver a abrirlo en pgAdmin 4 en el futuro, **pero no modifica la base de datos activa**.

5. El diccionario de datos

El **diccionario de datos (DD)** de una base de datos contiene información sobre la **estructura y organización de dicha base de datos**. Constituye lo que se llama una meta-base: una base de datos que describe a otra base de datos. El diccionario de datos suele contener la siguiente información:

- Los esquemas externos, el esquema conceptual e interno.
- Las descripciones de los registros, campos y relaciones a cada nivel.
- Los procedimientos de autorización de accesos y validación de datos.
- Referencias cruzadas del tipo:
  - Un programa qué datos utiliza
  - Un dato qué programas lo utilizan/modifican.

- Las correspondencias externa/conceptual y conceptual/interna.

El DD es utilizado por el SGBD, pero además es una valiosa herramienta tanto para el Administrador de la BD (le facilita el mantenimiento centralizado de la BD), como para usuarios, programadores y analistas de sistemas, para conocer con detalle la base de datos.

Actividad 8: Explorando el Diccionario de Datos en PostgreSQL"

En esta práctica, vamos a comprobar cómo el SGBD (en nuestro caso PostgreSQL) guarda en su **meta-base** (diccionario de datos) la información sobre esquemas, campos, permisos y correspondencias de la base de datos real.

No te preocupes si no entiendes algunos de los comandos que vamos a utilizar, los iremos viendo paso a paso durante el curso.

Preparamos el entorno creando una tabla Empleados y una vista sobre sus datos.

```
-- 1. Creamos la tabla del Nivel Conceptual
CREATE TABLE empleados (
    id_empleado SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    puesto VARCHAR(30),
    salario NUMERIC(8,2) CHECK (salario > 0)
);
-- 2. Creamos una Vista del Nivel Externo
CREATE VIEW vista_empleados_publica AS
SELECT id_empleado, nombre, puesto
FROM empleados;
```

El diccionario de datos almacena qué objetos existen y a qué nivel pertenecen.

a) Identificación de Esquemas y Objetos (Tablas y Vistas)

Ejecuta la siguiente consulta para consultar las tablas y vistas registradas en el catálogo del sistema:

```
SELECT
    table_name AS nombre_objeto,
    table_type AS tipo_objeto
FROM information_schema.tables
WHERE table_schema = 'public';
```

Haz captura de pantalla de la ejecución de la consulta. ¿Cómo distingue el diccionario de datos entre la tabla base (empleados) y la vista (vista_empleados_publica)?

b) Descripción de Registros y Campos (Metadatos de Columnas)

El **diccionario de datos** describe las propiedades físicas y lógicas de cada columna.

Consulta los metadatos de las columnas creadas ejecutando:

```
SELECT
    column_name AS columna,
    data_type AS tipo_dato,
    is_nullable AS permite_nulos
FROM information_schema.columns
WHERE table_name = 'empleados';
```

Haz captura de pantalla del resultado. Observa los resultados devueltos por el SGBD:

- a) ¿Qué tipo de dato registra el diccionario para el campo salario?
- b) ¿Qué valor toma is_nullable en la columna nombre? ¿Con qué regla DDL que escribimos al crear la tabla se corresponde?

c) Procedimientos de Autorización y Permisos de Acceso

El diccionario de datos registra qué permisos de acceso existen sobre cada objeto.

Consultemos los privilegios asociados a la tabla base mediante la tabla de catálogo `information_schema.table_privileges` mediante la siguiente consulta:

```
SELECT
    grantee AS usuario_o_rol,
    privilege_type AS tipo_permiso
FROM information_schema.table_privileges
WHERE table_name = 'empleados';
```

Haz captura de pantalla. ¿Qué usuario aparece registrado con permisos sobre la tabla `empleados` y qué tipos de acciones (`SELECT`, `INSERT`, …) tiene autorizadas?

d) Correspondencias y Referencias Cruzadas (Vista / Tabla Base)

El diccionario de datos mantiene la trazabilidad de qué vistas o consultas dependen de qué datos.

Ejecuta la siguiente consulta sobre el catálogo interno de PostgreSQL (pg_depend)

para ver qué relación de dependencia existe entre la vista externa y la tabla base:

```
SELECT
    viewname AS vista_externa,
    definition AS consulta_sql_origen
FROM pg_views
WHERE viewname = 'vista_empleados_publica';
```

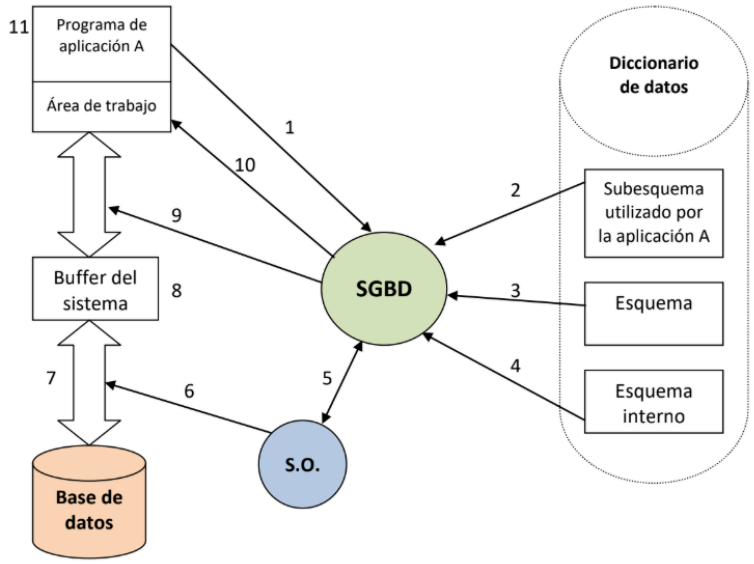
Haz captura de pantalla. ¿Cómo demuestra esta consulta que el diccionario de datos conoce la **correspondencia entre el nivel externo (la vista) y el nivel conceptual (la tabla base)**?

6. Funcionamiento de un SGBD

Para describir el funcionamiento de un SGBD utilizaremos su función más habitual, aquella en la que un programa de aplicación realiza una solicitud a un SGBD y si esta solicitud es aceptada, recibe el resultado de dicha solicitud. Los pasos serían:

a. El programa de aplicación A pide al SGBD que lea un registro. Se proporciona la clave de acceso. El SGBD evalúa la capacidad del usuario para realizar la operación solicitada.
b. El SGBD obtiene el subesquema que utiliza el programa de aplicación A. Si no encuentra los datos solicitados, rechaza la solicitud.
c. El SGBD obtiene el esquema y determina qué tipo de datos lógicos necesita.
d. El SGBD examina el esquema interno y determina qué registro físico debe leer.
e. El SGBD ordena al S.O. que lea el registro pedido.
f. El S.O. interacciona con el dispositivo físico en el que se encuentran los datos.
g. Los datos pedidos se envían desde el dispositivo físico al buffer del sistema.
h. El SGBD analiza el esquema, el subesquema y las correspondencias externa/conceptual para realizar las transformaciones necesarias de los datos.
i. El SGBD transfiere los datos al área de trabajo del programa de aplicación A.
j. El SGBD informa al programa del éxito o fracaso de la operación de E/S, incluyendo cualquier indicación de error.
k. El programa puede ahora trabajar con los datos pedidos.

El esquema podría ser parecido al siguiente:



Actividad 9: Refleja en un diagrama de flujo cómo sería el proceso.

7. Lenguaje SQL como medio de acceso a los sistemas gestores de bases de datos: LDD y LMD

Los SGBD relacionales utilizan **SQL** (structured query language) para su **descripción, manipulación y consulta**, que puede ser utilizado de modo interactivo o de modo procedural bien inmerso en cualquier lenguaje de programación o haciendo uso de extensiones que lo dotan de más potencia, como por ejemplo: PL-SQL de Oracle, TransactSQL de Microsoft SQL Server o **PL/pgSQL** (*Procedural Language/PostgreSQL Structured Query Language*) en PostgreSQL.

Aunque se trata de un único lenguaje, podemos clasificar su conjunto de instrucciones en dos grupos:

LMD (lenguaje de consulta y manipulación de datos), que incluiría sentencia de tipo:

- SELECT
- INSERT
- UPDATE
- DELETE

LDD (lenguaje de descripción de datos), que incluiría sentencia de tipo:

- CREATE
- ALTER

- DROP

Actividad 10: La Arquitectura de un Gimnasio: Separando el Contenedor (LDD) del Contenido (LMD)

En esta actividad, vamos a diferenciar las órdenes del **Lenguaje de Descripción de Datos (LDD / DDL)** y las del **Lenguaje de Manipulación de Datos (LMD / DML)**.

Vas a abrir un **Gimnasio**:

LDD (Lenguaje de Descripción de Datos): Es el trabajo de los **arquitectos y constructores**. Diseñan la estructura del edificio, definen dónde van los vestuarios, construyen las salas (CREATE), hacen modificaciones del espacio (ALTER) o tiran abajo una sauna que ya no se usa (DROP). **Afecta a la estructura del gimnasio.**

LMD (Lenguaje de Manipulación y Consulta de Datos): Es el trabajo del **recepcionista y los socios**. Dan de alta a un nuevo socio (INSERT), consultan la lista de inscritos a una clase (SELECT), cambian el número de teléfono de un usuario (UPDATE) o dan de baja a un socio que se cambia de ciudad (DELETE). **Afecta a los clientes dentro del gimnasio, no al edificio.**

a) PARTE 1: Clasificación de Órdenes SQL

Indica para cada una de las siguientes necesidades operativas de una empresa si se debe utilizar una orden **LDD** o **LMD**, y especifica el comando SQL concreto que le corresponde (CREATE, ALTER, DROP, SELECT, INSERT, UPDATE, DELETE):

1. Un cliente cambia de domicilio y necesitamos actualizar su dirección en la ficha.
2. La empresa decide añadir una nueva columna llamada email a la tabla de empleados.
3. El departamento de contabilidad solicita una lista de las facturas emitidas este mes.
4. Queremos eliminar por completo la tabla promociones_2020 porque ya no forma parte del modelo de datos.
5. Se registra una nueva venta realizada en el punto de cobro.
6. Se crea una nueva tabla llamada proveedores para guardar sus datos de contacto.
7. Un usuario cancela su cuenta y solicitó eliminar todos sus datos personales.

#	Necesidad operativa	Tipo (LDD / LMD)	Comando SQL
1	Actualizar dirección de un cliente	LMD	UPDATE
2	Añadir la columna email a la tabla		
3	Consultar lista de facturas emitidas		
4	Eliminar la estructura de la tabla promociones_2020		
5	Registrar una nueva venta		
6	Crear la estructura de la tabla proveedores		
7	Borrar la fila/registro de un usuario		

b) PARTE 2: Experimentación en PostgreSQL / pgAdmin 4

b.1) Abre **Query Tool** en pgAdmin 4 y ejecuta el siguiente bloque de comandos paso a paso. A medida que ejecutes cada instrucción, responde a las preguntas planteadas:

```
SQL
 -- PASO A:
 CREATE TABLE socios (
     id_socio SERIAL PRIMARY KEY,
     nombre VARCHAR(50),
     cuota NUMERIC(6,2)
 );
 -- PASO B:
 INSERT INTO socios (nombre, cuota) VALUES ('María López',
 45.00);
 INSERT INTO socios (nombre, cuota) VALUES ('Carlos Ruiz',
 50.00);
 -- PASO C:
 SELECT * FROM socios;
 -- PASO D:
 ALTER TABLE socios ADD COLUMN telefono VARCHAR(15);
 -- PASO E:
```

```
UPDATE socios SET cuota = 40.00 WHERE nombre = 'María López';
```

b.2) Responde a las siguientes preguntas:

- ¿Qué comandos del script pertenecen a **LDD** y cuáles a **LMD**?
- Tras ejecutar el **PASO D (ALTER TABLE)**, ¿se han borrado los datos de María y Carlos que insertaste en el paso B? Justifica cómo interactúan el LDD y el LMD.
- Si ejecutamos la orden DROP TABLE socios;
    - a) ¿Es una orden LDD o LMD?
    - b) ¿Cuál es la diferencia entre hacer DELETE FROM socios; y hacer DROP TABLE socios;?

c) PARTE 3: Reflexión sobre Extensiones Procedimentales (PL/pgSQL)

Imagina que el gimnasio establece la siguiente regla de negocio:

"Si un socio cumple más de 5 años de antigüedad, el sistema debe calcular automáticamente un 10% de descuento en su cuota, enviarle un correo de felicitación y registrarlo en la tabla de socios VIP."

Responde:¿Crees que con una sola instrucción interactiva de SQL estándar (SELECT, INSERT, UPDATE...) se puede programar toda esa lógica condicional y bucles en el servidor? ¿Por qué se hace necesario en estos casos el uso de extensiones como **PL/pgSQL** en PostgreSQL (o PL/SQL en Oracle)?

8. Modelo relacional: Estructura e integridad

El modelo relacional abarca tres aspectos clave: **estructura**, **integridad** y **manipulación**. De momento, nos centraremos en los dos primeros.

Estructura de datos relacional

Una base de datos relacional es, de forma sencilla, un conjunto de tablas interconectadas entre sí.

Desde el punto de vista del modelo relacional, cada tabla representa una **relación**. Una relación es un conjunto de **filas** (registros), donde cada fila está formada por varios **campos** (atributos) que guardan un valor concreto.

Para entender cómo se organiza una tabla, distinguimos dos conceptos clave:

- **Esquema:** Es la "plantilla" o estructura donde definimos qué campos tendrá la tabla y de qué tipo serán (el diseño).
- **Instancia:** Es la "fotografía" o el contenido real que tiene esa tabla en un

momento determinado.

Actividad 11: Esquema vs. Instancia

a) La analogía de los moldes de repostería

Antes de tocar el ordenador, responde a estas dos preguntas:

a.1) Si un repostero compra un **molde de silicona con forma de estrella**, ¿el molde es el **Esquema** o la **Instancia**?

a.2) Cuando hornea tres tartas distintas usando ese mismo molde el lunes, el martes y el miércoles, ¿cada una de esas tartas horneadas es un **Esquema** o una **Instancia**?

b) Práctica en PostgreSQL / pgAdmin 4

b.1) Abre el **Query Tool** de pgAdmin 4 y modifica la tabla Socios1 de la base de datos Gimnasio creada en el ejercicio anterior con las siguientes órdenes:

```
-- 1. Añade una nueva columna 'fecha_alta'
ALTER TABLE socios1 ADD COLUMN fecha_alta DATE;
-- 2. Renombra la tabla de 'socios1' a 'socios2'
ALTER TABLE socios1 RENAME TO socios2;
```

¿Qué has modificado, el esquema o la instancia?

b.2) Inserta los siguientes datos y consulta la tabla:

```
INSERT INTO socios (nombre, cuota, fecha_alta) VALUES ('María López', 45.00, '2024-01-15');
INSERT INTO socios (nombre, cuota, fecha_alta) VALUES ('Carlos Ruiz', 50.00, '2024-02-01');
SELECT * FROM socios;
```

Haz captura de pantalla de la tabla resultante. ¿Es esa tabla con 2 filas el **Esquema** o la **Instancia A** de la base de datos a las 10:00 AM?

b.3) Modificar los datos

Ejecuta ahora estas dos órdenes LMD:

```
DELETE FROM socios WHERE nombre = 'María López';
INSERT INTO socios (nombre, cuota, fecha_alta) VALUES ('Laura Gómez', 40.00, '2024-03-10');
```

```
SELECT * FROM socios;
```

¿Ha cambiado el **Esquema** de la tabla socios (sus columnas y tipos de datos)?

¿Sigue siendo la misma **Instancia** que la del Paso 2 o tenemos ahora una **Instancia B** distinta?

Conceptos Básicos del Modelo Relacional

- **Tupla (Fila / Registro):** Es cada uno de los elementos o filas que componen una tabla. El número total de tuplas se denomina **cardinalidad** (indica la cantidad de datos almacenados).
- **Atributo (Columna / Campo):** Es cada una de las columnas de la tabla y representa una característica del objeto. El número total de atributos se llama **grado** (indica la anchura de la tabla).
- **Dominio:** Es el conjunto de valores válidos que puede tomar un atributo. Por ejemplo, el dominio de la columna edad sería el conjunto de números enteros positivos permitidos.
- **Clave Candidata:** Es un atributo (o combinación de varios) que identifica de forma única e inconfundible a cada tupla. Debe cumplir dos reglas obligatorias:
  1. **Unicidad:** No existen dos filas con el mismo valor.
  2. **Minimalidad:** Si la clave consta de varios campos, no sobra ninguno. Si quitamos un campo y sigue identificando la fila sin duplicados, no era minimal.

     (Ejemplo: En una tabla de proveedores, la combinación (NIF, Localidad) identifica la fila, pero no es minimal porque con el NIF por sí solo basta).
- **Clave Primaria (Primary Key) y Claves Alternativas:** Cuando una tabla tiene varias *claves candidatas*, el diseñador elige una como **Clave Primaria** (la principal para identificar los registros). Las candidatas que no se eligen pasan a llamarse **Claves Alternativas**. *Toda tabla relacional debe tener siempre una Clave Primaria*.
- **Clave Ajena o Foránea (Foreign Key):** Es un atributo de una tabla que hace referencia a la *Clave Primaria* de otra tabla. Sirve para conectar y relacionar ambas tablas entre sí, exigiendo que ambos campos compartan el mismo dominio.

  (Ejemplo: El campo NIF_Proveedor en la tabla Facturas es una clave ajena que

apunta al NIF de la tabla Proveedores).

Actividad 12: Explica con algún ejemplo de base datos y de manera gráfica, qué elemento/s de esa base de datos se corresponderían con cada uno de los conceptos vistos en este apartado

Actividad 13: ¿Qué relaciones establecerías en las tablas de la actividad 7?
Haz captura de pantalla del modelo con las relaciones establecidas.

Actividad 14: Practicamos las restricciones de integridad en PostgreSQL
Comprobar cómo el SGBD actúa como un "filtro de seguridad" impidiendo la entrada de datos basura, duplicados o incoherentes mediante el uso de restricciones de integridad lógicas y referenciales.

Vamos a gestionar la base de datos de un **Centro de Formación**. Para ello, crearemos dos tablas relacionadas: **cursos y alumnos** .

PARTE 1: Construcción del Esquema con Restricciones (LDD)

Crea las tablas con las siguientes condiciones:

cursos

Campos:

id_curso entero y clave primaria

nombre_curso texto tamaño máximo 50, no puede ser nulo y no se puede repetir

precio numérico con 6 cifras (de las cuales 2 son decimales).

Añade una restricción que compruebe que el precion es >=0

alumnos

DNI texto tamaño 9 y clave primaria

nombre texto tamaño máximo 50 y no puede ser nulo

email texto tamaño máximo 100 y no se puede repetir

id_curso entero

Añade a esta tabla la restricción de FOREIGN KEY (clave foránea)

Insertamos datos de prueba:

```
INSERT INTO cursos VALUES (1, '1° ASIR', 0.00);
INSERT INTO cursos VALUES (2, '2° ASIR', 150.00);
INSERT INTO alumnos VALUES ('12345678A', 'Juan Pérez',
```

```
'juan@email.com', 1);
INSERT INTO alumnos VALUES ('87654321B', 'Ana Gómez',
'ana@email.com', 2);
```

PARTE 2: Comprobar que las restricciones actúan correctamente

Ejecuta los siguientes comandos SQL **uno por uno** (o de forma independiente) y haz captura de pantalla de los errores devueltos.

a) Restricción de Clave Primaria (PRIMARY KEY)

Intenta registrar un alumno con un DNI que ya existe:

```
INSERT INTO alumnos VALUES ('12345678A', 'Carlos López',
'carlos@email.com', 1);
```

¿Por qué rechaza PostgreSQL esta inserción? ¿Qué propiedad esencial de la Clave Primaria se estaría violando?

b) Restricción de Dominio y Valor Nulo (NOT NULL)

Intenta registrar un nuevo curso sin especificar su nombre:

```
INSERT INTO cursos (id_curso, precio) VALUES (3, 200.00);
```

¿Qué restricción se activa al omitir el nombre del curso?

c) Restricción de Chequeo (CHECK)

Intenta crear un curso con un precio negativo:

```
INSERT INTO cursos VALUES (4, 'Curso Python', -50.00);
```

¿Qué diferencia hay entre una restricción de tipo de dato (ej. NUMERIC) y una restricción CHECK?

d) Restricción de Integridad Referencial (FOREIGN KEY)

Intenta matricular a un alumno en un curso que no existe en la tabla cursos (ejemplo: curso ID 99):

```
INSERT INTO alumnos VALUES ('11112222C', 'María Soler',
'maria@email.com', 99);
```

¿Qué error nos da? ¿Cómo garantiza la integridad de los datos?

e) Eliminación de datos relacionados mediante clave foránea

Intenta borrar el curso 1º ASIR (ID 1) de la tabla cursos

```
DELETE FROM cursos WHERE id_curso = 1;
```

¿Por qué PostgreSQL nos impide borrar el curso 1? ¿Qué ocurriría con los alumnos de la tabla alumnos si la base de datos permitiera borrarlo sin control?

f) Aunque lo hablaremos en puntos posteriores, ¿qué opciones nos ofrece SQL cuando tratamos de eliminar datos que están vinculados con valores de otras tablas mediante clave foránea?

10. Integridad de los datos

En una base de datos relacional, las **relaciones están sujetas a ciertas reglas de integridad:**

Integridad de entidad: "Ningún atributo que participe en una clave principal puede tener valores nulos."

Codd[1] lo justifica esto del siguiente modo: En el modelo relacional, **las relaciones/tablas representan entidades del mundo real y todas las entidades del mundo real son distinguibles**. Las **claves primarias llevan a cabo la función de identificación en el modelo relacional**, por tanto un identificador nulo sería una contradicción.

Integridad referencial: "No deben existir valores de clave ajena sin concordancia."

Dicho de otro modo, **si una relación/tabla tiene una clave ajena que referencia a otra relación (o a ella misma), cada valor de la clave ajena tiene que ser igual a un valor de la clave principal de la relación a la que referencia, o bien, ser completamente nulo.**

Para mantener la integridad de una base de datos, el diseñador de la misma deberá especificar qué operaciones se rechazarán y cuáles se aceptaran, y en este caso, qué operaciones de compensación habrá que realizar.

Para ello el diseñador debe plantearse **tres preguntas por cada clave ajena:**

1. ¿Puede aceptar nulos esa clave ajena?

Ejemplo: Tenemos tres tablas en nuestro modelo de base de datos, PROVEEDORES, ARTICULOS y PRECIOS, ¿tiene sentido la existencia de un precio de un artículo que no está vinculado a ningún proveedor? En este caso no, pero en algunos casos como EMPLEADOS y PROYECTOS, podría existir un empleado no asignado de momento a un

[1] Matemático y científico informático británico conocido por inventar el **modelo relacional de bases de datos**

proyecto.

2. ¿Qué deberá suceder si se intenta eliminar una tupla referenciada por una clave ajena?

Ejemplo: Se intenta eliminar un proveedor del cual existe algún precio de artículo. En general, para estos casos existen por lo menos tres posibilidades:

Restringir: La operación de eliminación se restringe sólo al caso en el que no existe alguna tupla con clave ajena que la referencie, rechazándose en caso contrario. En nuestro ejemplo, un proveedor podrá ser borrado, si y sólo si, no tiene precios de artículos, es decir, no suministra artículos.

Propagar en cascada: La operación de borrado se propaga en cascada eliminando también todas las tuplas cuya clave ajena la referencien. En nuestro ejemplo, se eliminaría el proveedor y todas las tuplas de precios de artículos suministrados por él.

Anular: Se asignan nulos en la clave ajena de todas las tuplas que la referencien y se elimina la tupla referenciada. En nuestro ejemplo, no tiene mucho sentido, pero consistiría en asignar nulos al NIF-PROV de todas las tuplas de precios pertenecientes al proveedor que queremos borrar, y posteriormente borrar al proveedor.

3. ¿Qué deberá suceder si hay un intento de modificar la clave primaria de una tupla referenciada por una clave ajena?

Ejemplo: Se intenta modificar la clave de un proveedor del cual existe algún precio de artículo. En general, para estos casos también existen por lo menos tres posibilidades:

Restringir: La operación de modificación se restringe sólo al caso en el que no existe alguna tupla con clave ajena que la referencie, rechazándose en caso contrario. En nuestro ejemplo, la clave de un proveedor podrá ser modificada, si y sólo si, no tiene precios de artículos, es decir, por el momento no suministra artículos.

Propagar en cascada: La operación de modificación se propaga en cascada modificando también la clave ajena de todas las tuplas que la referencien. En nuestro ejemplo, se modificaría el NIF del proveedor y la clave ajena NIF-PROV de todos los precios de artículos suministrados por él.

Anular: Se asignan nulos en la clave ajena de todas las tuplas que la referencien y se modifica la clave de la tupla referenciada. En nuestro ejemplo, tampoco tiene mucho sentido, pero consistiría en asignar nulos al NIF-PROV de todas las tuplas de precios

pertenecientes al proveedor que queremos modificar, y posteriormente modificar la clave del proveedor.

Las opciones vistas de restringir, propagar en cascada y anular, sólo representan un conjunto de casos que se presentan con bastante frecuencia en la práctica. Pero podrían adoptarse otras soluciones. Por ejemplo, para un intento de borrado de un proveedor:

- Podría efectuarse un diálogo con el usuario.
- Podría grabarse información en otro archivo de antiguos proveedores.
- Podría transferirse a otro proveedor los artículos del proveedor en cuestión.
- …

Actividad 15: Analiza las siguientes **3 situaciones reales** de bases de datos. Para cada clave ajena implicada, responde a las **3 preguntas del diseñador** planteadas en la teoría y justifica cuál de las políticas (RESTRICT, CASCADE, SET NULL) es la más adecuada en cada caso.

Caso A: Hospital (Pacientes y Médicos Cabecera)

- **Tabla Padre:** `medicos (id_medico PRIMARY KEY, nombre)`
- **Tabla Hija:** `pacientes (dni PRIMARY KEY, nombre, id_medico FOREIGN KEY)`

a) ¿Debe el campo id_medico de la tabla pacientes permitir valores nulos? *(Pensad si un paciente recién registrado puede estar temporalmente sin médico asignado)*.

b) Si un médico se jubila y se borra de la tabla medicos, ¿qué debería ocurrir con los registros de sus pacientes? ¿Se borran en cascada, se anulan (SET NULL) o se restringe el borrado?

c) Si cambia el id_medico de un facultativo, ¿qué política debe aplicarse sobre sus pacientes?

Caso B: Tienda Online (Pedidos y Líneas de Detalle)

- **Tabla Padre:** `pedidos (id_pedido PRIMARY KEY, fecha, cliente)`
- **Tabla Hija:** `lineas_pedido (id_pedido FOREIGN KEY, num_linea, producto, cantidad)`

a) ¿Puede el campo id_pedido en lineas_pedido permitir valores nulos?

b) Si un cliente cancela un pedido y borramos la fila en pedidos, ¿qué debe suceder con sus líneas de detalle en lineas_pedido? Justifica por qué RESTRICT o SET NULL

serían errores graves en este caso.

c) Si el número de identificación de un pedido se modifica, ¿qué debe ocurrir con sus líneas?

Caso C: Matrículas del Gimnasio (Socios y Clases)

- **Tabla Padre:** socios (id_socio PRIMARY KEY, nombre)
- **Tabla Hija:** asistencias (id_socio FOREIGN KEY, id_clase FOREIGN KEY, fecha)

a) Si un socio decide darse de baja de la base de datos por la ley de protección de datos (LOPD) y se elimina de socios, ¿qué ocurre con el historial de asistencias a las clases?

b) Proponed una **solución de compensación** diferente a las tres estándar (por ejemplo: archivar la información o transferir los datos).

El Borrado en Cascada en Cadena (Efecto Dominó)

Imagina una base de datos con **3 tablas conectadas en cadena**:

Empresas (Ficha 1)

Departamentos (Ficha 2: pertenecen a una empresa)

Empleados (Ficha 3: pertenecen a un departamento)

Si pides al sistema **borrar una Empresa**:

Nivel 1: El sistema borra esa empresa. Como está en cascada, pasa a borrar todos los **departamentos** de esa empresa.

Nivel 2: Al intentar borrar esos departamentos, el efecto en cascada continúa y pasa a borrar a todos los **empleados** que trabajaban en esos departamentos.

La regla del "Todo o Nada" (Transacción)

Muy importante: **la cadena no se puede romper a la mitad**.

Si el borrado de departamentos en cascada intenta eliminar a un empleado, **pero la tabla Empleados tiene una regla que prohíbe el borrado** (por ejemplo, porque el empleado tiene facturas firmadas a su nombre), **toda la operación se cancela.**

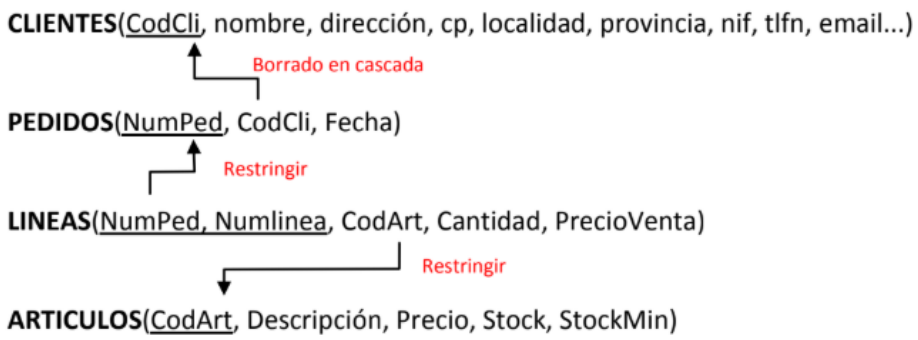
No se borra el empleado, **tampoco se borran los departamentos y tampoco se borra la empresa**. La base de datos vuelve a su estado original exactamente como si nadie hubiera tocado nada. O se borra la cadena entera con éxito, o se aborta por completo.

¿Y en las modificaciones?

Ocurre lo mismo. Si cambias el identificador de la empresa (su código o NIF) y la opción en cascada está activada, ese nuevo código se actualizará automáticamente en sus departamentos y a su vez, en las fichas de todos sus empleados.

El borrado en cascada multinivel es una reacción en cadena. Si en cualquier punto de la cadena una regla falla o impide el borrado, **toda la operación se da marcha atrás (*rollback*)** para evitar que la base de datos quede a medias con datos corruptos.

Actividad 16: Veamos un ejemplo, suponemos las siguientes relaciones/tablas con los criterios de integridad referencial indicados:



a) Podemos eliminar un cliente si tiene algún pedido pendiente o línea de pedido asociada?

Sólo se eliminaría si no tiene pedidos pendiente, o si tiene algún pedido sin líneas de pedido asociadas.

b) Identifica la clave principal y ajena de cada una de las tablas.

c) ¿Tiene sentido que la clave ajena CodCli de la tabla PEDIDOS pueda tomar valores nulos? Independientemente de si tiene o no sentido. ¿Estaría permitido establecer que admitan valores NULOS dicha la clave ajena? ¿Por qué?

d) ¿Tiene sentido que la clave ajena NumPed de la tabla LINEAS pueda tomar valores nulos? Independientemente de si tiene o no sentido. ¿Estaría permitido establecer que admitan valores NULOS dicha la clave ajena? ¿Por qué?

Una línea de detalle no puede existir "flotando" sin pertenecer a la cabecera de un pedido concreto.

d) Si queremos que al borrar un cliente se borren todos sus pedidos junto con sus

líneas de pedido. ¿Qué restricciones de integridad referencial tendríamos que establecer?

e) Si suponemos que todas las restricciones que aparecen en el esquema anterior son del tipo borrado en cascada:

Al borrar un cliente se eliminarían los pedidos y sus líneas. ¿Se borrarían también los artículos a los que referencian dichas líneas?

Al borrar un artículo, se borrarían las líneas que lo referencian. ¿Se borrarían también los pedidos a los que pertenecen esas líneas junto con los clientes?